МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Лабораторная работа 3. Обработка строк

Студент
гр. БИН-26-1 _____ И.Д. Плющиков

Преподаватель _____ М.В. Водяницкий

Владивосток 2026

1 Выполнение работы

1.1 Задание 1. Создание кода документа

В ходе выполнения задания предусмотрен ввод кода документа в формате AAA-NNNNNNNN. Первая часть кода представлена тремя заглавными латинскими буквами и обозначает категорию, вторая содержит цифры года, третья включает четыре цифры номера.

Изображение алгоритма для решения первого задания 1.

```
1   code = input()
2
3   cut_1 = code[:3]
4   cut_2 = code[4:8]
5   cut_3 = code[9:13]
6   cut_4 = cut_3[::-1]
7
8   print(f'Категория: {cut_1}')
9   print(f'Год: {cut_2}')
10  print(f'Номер: {cut_3}')
11  print(f'Обратный номер: {cut_4}')
12
```

Рисунок 1 – Текст программы для первого задания

Требовалось с помощью срезов изменить внешний вид, год и номер, при этом сохранение полученных результатов не выполнялось. Дополнительно необходимо вывести номер в обратном порядке. Применение срезов позволяет извлекать нужные фрагменты строки по индексам, что упрощает и делает более наглядной обработку кода документа.Изображение результата выполнения программы.

Изображение результата выполнения программы 2.

```
RUS-2026-0042
Категория: RUS
Год: 2026
Номер: 0042
Обратный номер: 2400
```

Рисунок 2 – Результат выполнения программы для первого задания

На этом скриншоте показан результат обработки кода документа: программа корректно выделила категорию, год и номер, а также вывела номер в обратном порядке. Срезы строки позволили получить нужные части без изменения исходных данных

1.2 Задание 2. Форматирование имени

В рамках выполнения задания пользователь вводит фамилию, имя и отчество одной строкой, при этом регистр букв может быть произвольным. Каждая часть строки состоит только из букв, между частями допускается один или несколько пробелов, а также возможны пробелы в начале и в конце строки

Изображение алгоритма для решения второго задания 3.

```
name = input()
last_name, first_name, middle_name = name.split()
result = f"{last_name.capitalize()} {first_name[0].upper()}. {middle_name[0].upper()}."
print(result)

```

Рисунок 3 – Текст программы для второго задания

Требовалось вывести фамилию, приведя первую букву к верхнему регистру, а остальные — к нижнему, после чего вывести инициалы имени и отчества в верхнем регистре. При обработке строки лишние пробелы необходимо игнорировать, а регистр букв привести к единому виду.

Изображение результата выполнения программы 4.

```
  иВАНОВ иВАН иВАНОВИЧ
Иванов И. И.
```

Рисунок 4 – Результат выполнения программы для второго задания

Здесь видно, как после обработки введённой строки фамилия приведена к нормальному регистру, а имя и отчество сокращены до инициалов. Лишние пробелы и смешанный регистр были успешно проигнорированы.

1.3 Задание 3. Очистка номера телефона

В рамках выполнения задания пользователь вводит номер телефона в формате +7 (NNN) NNN-NN-NN, где каждый символ N обозначает цифру от 0 до 9.

Изображение алгоритма для решения третьего задания 5.

```
numb = input()
numb = numb.replace(' ', '').replace('+', '').replace('(', '').replace(')', '').replace('-', '')
print(numb)

```

Рисунок 5 – Текст программы для третьего задания

Требовалось удалить из введённой строки символы «+», пробелы, круглые скобки и дефисы. В результате должна остаться только последовательность цифр, пригодная для дальнейшей обработки или хранения.

Изображение результата выполнения программы 6.

```
+7 (924) 123-45-67
79241234567
```

Рисунок 6 – Результат выполнения программы для третьего задания

Результат работы программы подтверждает, что из номера телефона удалены все лишние символы: плюс, пробелы, скобки и дефисы. В строке остались только цифры, что удобно для дальнейшей обработки.

1.4 Задание 4. Разбор записей о поезде

В рамках выполнения задания пользователь вводит сведения о поезде одной строкой, поля которой разделены символом «;» в следующем порядке: номер, пункт отправления, пункт назначения, время, цена. Все пять полей должны быть непустыми, не содержать символа «» и пробелов по краям; внутри названий городов пробелы допустимы. Время задаётся в формате HH:MM, цена — неотрицательное число, в котором десятичным разделителем является точка.

Изображение алгоритма для решения четвертого задания 7.

```
1  no_sort = input()
2  parts = no_sort.split(";")
3  price = float(parts[4])
4  print(f'Поезд: {parts[0]}')
5  print(f'Маршрут: {parts[1]} - {parts[2]}')
6  print(f'Отправление: {parts[3]}')
7  print(f'Цена: {price:.2f} руб')
8
```

Рисунок 7 – Текст программы для четвертого задания

Требовалось разобрать введённую строку на отдельные значения и вывести официальную запись. Цена при этом выводится с двумя цифрами после десятичной точки. Для разбора строки удобно использовать метод split() с разделителем «;», а для форматирования цены — форматированную строку

Изображение результата выполнения программы 8.

```
Т-18;Владивосток;Хабаровск;08:45;1850.5
Поезд: Т-18
Маршрут: Владивосток - Хабаровск
Отправление: 08:45
Цена: 1850.50 руб
```

Рисунок 8 – Результат выполнения программы для четвертого задания

В консоли выведена официальная запись о поезде с номером, маршрутом, временем отправления и ценой. Цена представлена с двумя знаками после десятичной точки, как и требовалось в задании.

1.5 Задание 5. Проверка идентификатора

В ходе выполнения задания необходимо проверить идентификатор, вводимый пользователем в виде одной строки. Строка может быть пустой или содержать пробелы. Проверка осуществляется над исходным текстом без удаления, замены или иного изменения символов.Требуется вывести длину строки и результаты следующих проверок: состоит ли строка только из букв, только из цифр, является ли она буквенно-цифровой, а также содержит ли она дефис. Буквенно-цифровой считается непустая строка, каждый символ которой является буквой или цифрой.

Изображение алгоритма для решения пятого задания 9.

```
1   code = input()
2   print(f'Длина: {len(code)}')
3   print(f'Только буквы: {code.isalpha()}')
4   print(f'Только цифры: {code.isdigit()}')
5   print(f'Буквенно-цифровая: {code.isalnum()}')
6   print(f"Содержит дефис: {'-' in code}")
7
```

Рисунок 9 – Текст программы для пятого задания

Для первых трёх проверок используются методы isalpha() и isdigit(), для четвёртой — isalnum(). Если строка пустая, её длина равна 0, а все четыре проверки возвращают False. Такой подход позволяет оценить состав строки без её модификации.

Изображение результата выполнения программы 10.

```
AB-204
Длина: 6
Только буквы: False
Только цифры: False
Буквенно-цифровая: False
Содержит дефис: True
```

Рисунок 10 – Результат выполнения программы для пятого задания

На скриншоте отражены длина строки и результаты проверок на состав символов. Исходный идентификатор не изменялся, а все проверки выполнены строковыми методами.

1.6 Задание 6. Формирование пути

В рамках выполнения задания пользователь вводит строку, содержащую ровно три непустые части пути, разделённые запятыми без пробелов. Части пути не должны содержать символы / и .

Изображение алгоритма для решения шестого задания 11.

```
1   project = input()
2   project1 = project.split(',')
3   result = '/'.join(project1)
4   print(result)
```

Рисунок 11 – Текст программы для шестого задания

Необходимо разделить введённую строку на составляющие с помощью метода split(), а затем объединить полученные элементы символом / с использованием метода join(). В результате выполнения программы должен быть получен корректный относительный путь, состоящий из трёх уровней.

Изображение результата выполнения программы 12.

```
projects,python,lab3
projects/python/lab3
```

Рисунок 12 – Результат выполнения программы для шестого задания

Здесь показан итоговый путь, собранный из трёх частей с разделителем /. Методы split() и join() позволили разделить строку и соединить её элементы в корректный относительный путь.